QA AUTOMATION CAPSTONE: PROJECT DESCRIPTION

PROJECT NAME

Enterprise E-Commerce UI Automation Framework using Playwright and Java Script.

PROJECT OVERVIEW

The Automation Exercise UI Automation Framework using Playwright and JavaScript is designed to validate the core functionalities of the Automation Exercise e-commerce platform. The framework covers critical business workflows including user authentication, home page validation, product search, cart management, checkout, shipping, and API testing. Built using the Page Object Model (POM) design pattern and environment-based configuration through .env files, the framework ensures high maintainability, scalability, and reusability.

The solution supports automated end-to-end testing across modern browsers, enabling consistent validation of user journeys and business processes. Detailed execution insights are generated through Allure Reports, including test results, execution trends, screenshots, and failure analysis. By combining UI automation and API validation within a single framework, the project provides reliable regression coverage, faster feedback cycles, and improved software quality, making it suitable for continuous testing and modern QA practices.

WEBSITE NAME

AUTOMATION EXCERCISE

WEBSITE URL

https://automationexercise.com/

PROJECT OBJECTIVES

- Prepare a complete test plan for the selected website.
- Create a minimum of 120 test cases across services.
- Cover positive, negative, validation, navigation, and regression scenarios.
- Automate the planned tests using Playwright with JavaScript.
- Generate execution evidence through test reports, screenshots, and traces wherever required.
- Present the project as a QA/testing capstone, not as a development project.

TEST CASE DISTRIBUTION

- **Services:** 7
- **Test cases per service:** ~18
- **Total per browser:** 126
- **Browsers:** Chromium, Firefox, WebKit
- **Grand Total Executions:** 378

PLANNED SERVICES

S.NO	Service / Module	Example Testing Coverage	Planned Test Cases
01.	**Authentication**	Valid/invalid login, logout, password reset, account creation, duplicate email, session timeout, lockout, remember-me, protected page access	18
02.	**Home**	Home page load validation, navigation menu verification, category visibility, featured products display, recommended products section, subscription functionality, banner/slider validation, footer links verification, responsive layout checks, page performance validation	18
03.	**Search & Discovery**	Add/remove/update items, multiple products, invalid quantity, max quantity, subtotal/tax/total calculation, coupon handling, persistence, cart icon count, cart clears after checkout	18
04.	**Cart Management**	Add product to cart, remove product from cart, update quantity, multiple product handling, cart total calculation, cart icon count validation, cart persistence after refresh, duplicate product handling, invalid quantity validation, continue shopping functionality	18
05.	**Checkout**	Proceed to checkout, order summary validation, billing information verification, product quantity validation, checkout without login validation, payment page navigation, coupon application validation, order confirmation verification, checkout cancellation, final amount validation	18
06.	**Shipping**	Shipping address validation, add/edit shipping address, mandatory field validation, delivery information verification, shipping method selection, shipping cost calculation, multiple address handling, invalid postal code validation, address persistence, order delivery summary validation	18
07.	**API Service (Mocking & Backend Validation)**	GET request validation, POST request validation, PUT request validation, DELETE request validation, response status code verification, response schema validation, response time validation, unauthorized access handling, error response validation, mock API data verification	18

CONCLUSION

This project delivers a robust and scalable automation framework for the Automation Exercise platform, ensuring reliable validation of critical user workflows and API functionalities. By combining Playwright, Page Object Model (POM), environment-based configuration, and Allure Reporting, the framework improves test coverage, reduces manual effort, and enhances software quality. It serves as a strong foundation for continuous testing and demonstrates best practices in modern test automation.